

Martian Corporation

INTERNSHIP PROJECT

AI-Based CCTV People Tracking & Dwell-Time Analytics System

Working Guidelines & Project Requirements

This document has two parts. **Part 1** explains how we would like you to work with us throughout the project. **Part 2** defines the project itself — the problem to solve, the goals, and the required functionality.

We are giving you a business requirement, the way a client gives a requirement to a development team: *this is what we need the system to do*. The technical approach — architecture, codebase structure, technology choices, project phases, milestones, and implementation plan — is for you to research, propose, defend, and own.

Part 1 — Working Guidelines & Principles

The way you work matters to us as much as the final result. How you research, communicate, make decisions, and document your progress is a core part of this internship. Please read this part carefully — these are the expectations we will hold consistently, and following them will make the project run smoothly for everyone.

1.1 How We Work Together (Client and Team)

- We act as the **client**. We provide the requirements, goals, expected use cases, and required functionality.
- You act as the **development team**. You research and propose the technical solution — the approach, architecture, how the codebase and project are organized, the technologies and tools, how the work is broken into phases and milestones, and the implementation, testing, and deployment plan.
- We will not hand you a ready-made solution. The design and technical decisions are yours to propose and justify, based on the requirements.
- Please do not begin development before the requirements are well understood and your proposed approach has been presented and discussed with us.

1.2 Daily Work Expectations

- There are **two short meetings each working day**. The first thing in every meeting is a status update, after which the next steps are discussed.

- Before each meeting, please have a brief written update ready.
- Keep all your work in the shared project repository, and keep it up to date the same day you do the work.

1.3 Communication & Progress Reporting

Each update — written before the meeting, and again at the end of the day before you log off — should briefly cover:

- What you worked on and completed.
- What you researched and learned.
- What problems you encountered.
- What decisions you made, and the reasoning behind them.
- What is still pending.
- What you plan to do next.

1.4 Honesty in Reporting

Please always report accurately. If a task could not be completed, simply tell us what happened — for example, *"I could not finish this because of X."* An honest update about a genuine blocker is far more valuable to us than one that looks complete but isn't accurate. We will never treat an honestly reported problem as a failure; unclear or inaccurate reporting is the only thing that causes difficulty.

1.5 Research & Decision-Making

- **Research before asking.** When you face a question, first think it through and research it — official documentation, GitHub, research papers, technical videos, and AI tools are all fair to use. Compare approaches and test your assumptions.
- When you do bring a problem to us, bring your findings with it: what you tried, what worked and what didn't, the options you found, and what you would recommend — rather than only *"I'm stuck."*
- **Validate what you find.** Information from AI tools and the internet should be checked, not accepted at face value.
- **Justify your technical choices.** For any significant technology, tool, or model you propose, be ready to explain what it is, why you chose it, the alternatives you considered, the trade-offs, and its implications for accuracy, performance, scalability, cost/licensing, and real-world use.

1.6 Documentation & Version Control

- All work — code, documents, research notes, references, and decisions — lives in the shared project repository (version-controlled).
- Document as you go, so that anyone can look at the project and follow what was done, and why, from one day to the next.
- Record your research, your decisions and the reasons for them, the problems you hit, and your progress.

1.7 Ownership & Collaboration

- Take ownership of your assigned work end to end: **Understand → Research → Think → Experiment → Validate → Document → Implement → Test → Report.**
- Both of you should take part actively in research, discussion, and documentation, and each of you is expected to think and propose independently rather than relying on the other.

1.8 Weekly Rhythm (Fridays)

- **Weekly summary:** every Friday, prepare a short summary of the week — what you accomplished, what you learned, what you researched, what challenges you faced and how you solved them, any mistakes and what you took from them, and what you plan for next week. The focus is on learning and problem-solving, not only on output.
- **LinkedIn post:** every Friday, share a short, professional post about something you learned that week (a concept, an approach, a problem you solved). Tag **Martian Corporation** where appropriate. Please do **not** include any confidential information — no private data, credentials, customer information, CCTV footage, or proprietary implementation details.

1.9 Summary of Expectations

1. Provide a brief written update before each meeting.
2. Report accurately and honestly; if something isn't done, tell us the real reason.
3. Research independently before asking for a solution, and bring findings and options when you do ask.
4. Keep all work in the shared, version-controlled repository, updated daily.
5. Document your research, decisions, problems, and progress as you go.
6. Justify every significant technical decision with clear reasoning.
7. Submit a weekly summary each Friday, and share an appropriate learning post on LinkedIn.
8. Never expose confidential or proprietary company information publicly.
9. Understand the requirements and propose an approach before starting development.
10. Both of you participate fully, and each proposes independently.
11. Progress steadily and thoughtfully rather than rushing to finish.

Part 2 — Project Definition & Requirements

This part defines *what* we need. It describes the problem, the goals, and the functionality the system must provide. How to achieve it is for you to research and propose.

2.1 Project Summary

We need an **AI-based CCTV monitoring and people-tracking system.**

A location has **multiple CCTV cameras** covering different areas. We need a system that uses those camera feeds to **detect, count, track, and analyze the people** moving through the premises, and makes the results available to an administrator.

2.2 The Business Need

Locations with many cameras cannot easily answer basic but valuable questions such as: *How many people came in? How many are here right now? How long did they stay? How did they move through the space?* The footage exists, but the insight does not. We need a system that turns the camera feeds into clear, usable information about people and their movement.

2.3 Where It Will Be Used & Who Uses It

- **Environments:** warehouses, showrooms, shopping malls, retail stores, large commercial spaces, or any location with multiple CCTV cameras and significant foot traffic.
- **Primary user:** an **administrator**, who needs to view and understand the results (rather than watch raw footage).

2.4 Goals & Expected Outcomes

For a given location, the system should be able to answer:

- How many people **entered** the premises.
- How many people are **currently present**.
- How many people have **visited in total**.
- How people are **distributed across different areas**.
- For any individual, **how long they stayed** and **how they moved** through the location.

2.5 Functional Requirements (Use Cases)

A. People Counting

The system must determine how many people entered, how many are currently present, how many have visited in total, and how many are in different areas. *Example:* if 80 people enter a mall during a period, the system should reflect and maintain that count.

B. Consistent Individual Identity Across Cameras

This is the most important requirement. An individual must keep the **same identity as they move between different cameras**. *Example:* a person who enters on Camera 1 is identified as **Person 001**; when they move to Camera 2, they must still be **Person 001**, not a new identity such as Person 002. *How* to achieve this across cameras is for you to research and propose.

C. Dwell-Time / Time-Spent Tracking

The system must measure how long an individual spends inside the premises, including entry time, exit time, total time spent, movement between areas, and — where possible — time spent in specific areas. *Example:* Person 001 → 4 hours; Person 002 → 20 minutes; Person 003 → 1 hour 15 minutes.

D. Multi-Camera Coverage

The system must work across all cameras in the location. A person moving through several cameras (for example, Camera 1 → Camera 3 → Camera 5 → Camera 2) must remain the same unique identity throughout their journey.

E. Information for the Administrator

The results must be made available to an administrator. The kinds of information an administrator needs include total visitors, current visitors, individual tracking, time spent, camera-wise and area-wise counts, historical data, analytics, and camera status — and alerts, if appropriate. *How* this is presented is for you to design and propose.

2.6 Non-Functional Requirements

The system is expected to satisfy these qualities. Consider each when you propose your approach:

- **Accuracy** of counting, tracking, and identity across cameras.
- **Performance** — how close to real time the system needs to operate.
- **Scalability** — behaviour as the number of cameras, people, and time period grows.
- **Reliability** — sensible behaviour when a camera or feed drops.
- **Data storage & retention** — what is stored and for how long.
- **Security & privacy** — footage and movement data are sensitive and must be handled responsibly.
- **Cost & licensing** — practical and appropriate for real-world use.
- **Real-world suitability** — the solution must work under realistic conditions, not only in ideal cases.

2.7 What the Team Will Propose & Own

From our side, the requirements above are what we are defining. The following are for **you** to research, decide, propose, and justify — we are intentionally leaving these open:

- The overall technical approach and system architecture.
- How the codebase and project are structured and organized.
- The technologies, libraries, models, and tools to use (with your reasoning).
- The database design and what data is stored.
- The design of what the administrator sees.
- How you divide the project into **phases and milestones**, and why.
- Your implementation, testing, and deployment plan.

We expect you to first understand these requirements, research thoroughly, and come back with your proposed approach and plan — which we will then review and discuss together.